# Redes neuronales (versión del profesor)

Tomas Santana
November, 2026

## Índice

1. Parte 1: Una neurona ..... 2
   1.1. De la regresión logística a una neurona ..... 2
   1.2. Funciones de activación ..... 2
   1.3. El límite de una neurona: XOR ..... 3
2. Parte 2: Redes neuronales ..... 4
   2.1. Apilar neuronas ..... 4
   2.2. La capa oculta transforma los datos ..... 5
   2.3. Redes totalmente conectadas ..... 5
   2.4. Notación vectorial ..... 6
   2.5. Redes con más capas ..... 8
   2.6. ¿Por qué no usar la identidad como activación? ..... 8
   2.7. La red aprende las features ..... 9
   2.8. Varias clases: softmax ..... 10
   2.9. (Opcional) Vectorización sobre varios ejemplos ..... 10
3. Parte 3: Cómo aprende una red ..... 11
   3.1. La pérdida ..... 11
   3.2. Descenso por gradiente ..... 11
   3.3. Backpropagation: el error viaja hacia atrás ..... 13
   3.4. Entrenar una red en PyTorch ..... 14
   3.5. (Opcional) Backpropagation en general ..... 14
4. Resumen ..... 15

### Plan de la clase (3 horas)

> - **0:00–0:55 · Bloque 1. Parte 1 y secciones 2.1 a 2.3. Ejercicios 1 y 2 en papel, en parejas (unos 25 min en total).**
> - **0:55–1:05 · Descanso.**
> - **1:05–1:55 · Bloque 2. Secciones 2.4 a 2.8 (unos 20 min; la 2.9 solo si van bien de tiempo). Demo del Playground (unos 10 min). Lab, Parte 1 (ejercicios 1 a 3 del notebook, unos 20 min).**
> - **1:55–2:50 · Bloque 3. Parte 3. Ejercicios 3 y 4 en papel. Lab, Parte 2 (ejercicios 4 y 5 del notebook).**
> - **2:50–3:00 · Cierre. Las tres ideas del día (Resumen) y presentar la Tarea 4.**
> - **Si se acaba el tiempo: la sección 2.9, la 3.5 y el ejercicio 6 del notebook son lo primero que se corta. No cortar el ejercicio 4 del notebook (`loss.backward()` da −32): es el momento clave del Bloque 3.**
> - **Recordar: el Playground no funciona sin internet en el laboratorio. Abrirlo antes de clase desde tu máquina.**

# 1. Parte 1: Una neurona

> **Bloque 1 (0:00). Arrancar con la pregunta: "¿Qué hicimos en la clase 4 cuando la frontera no era una recta?" Que respondan ellos: inventamos features, como $x_1^2$. Esa es la motivación de toda la clase: hoy la red las inventa sola.**

## 1.1. De la regresión logística a una neurona

Recordemos el modelo de regresión logística de la clase 4. Dado un ejemplo $\mathbf{x} \in \mathbb{R}^d$, el modelo calcula un score lineal $\theta^T \mathbf{x}$ y lo pasa por la función sigmoide:

$$h_\theta(\mathbf{x}) = g(\theta^T \mathbf{x}), \qquad g(z) = \frac{1}{1+e^{-z}}.$$

En redes neuronales es costumbre escribir el intercepto por separado, en lugar de agregar una feature $x_0 = 1$. Escribimos los parámetros como un vector de pesos $\mathbf{w} \in \mathbb{R}^d$ y un número $b \in \mathbb{R}$, que llamamos el sesgo (bias). El score es entonces

$$z = \mathbf{w}^T \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b,$$

que es lo mismo que $\theta^T \mathbf{x}$ con $\theta_0 = b$ y $\theta_j = w_j$ para $j = 1, \ldots, d$. La predicción es

$$a = g(z).$$

A una unidad que calcula una suma ponderada de sus entradas, le suma un sesgo y aplica una función no lineal al resultado, la llamamos una **neurona**. La función $g$ se llama **función de activación**, y el número $a$ es la **activación** de la neurona. Con esta terminología, la regresión logística es una red neuronal con una sola neurona (Figura 1).

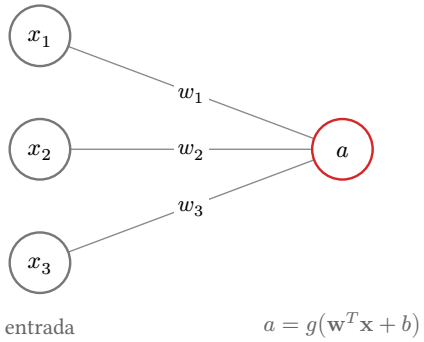



Figura 1: La regresión logística dibujada como una neurona. Cada entrada $x_j$ se multiplica por su peso $w_j$, se suman los resultados y el sesgo $b$, y la suma $z$ pasa por la activación $g$.

El nombre viene de las neuronas del cerebro: una neurona biológica recibe señales de otras, y se activa si la señal total es suficientemente fuerte. La analogía es bastante libre, y no hay que tomarla muy en serio. Para nosotros, una neurona es simplemente una función de $\mathbf{x}$ con parámetros $\mathbf{w}$ y $b$.

## 1.2. Funciones de activación

La sigmoide no es la única función de activación posible. Supongamos, por ejemplo, que queremos predecir el precio de una casa a partir de su tamaño. Una recta $wx + b$ puede dar precios negativos para casas muy pequeñas. Una forma de evitarlo es poner el precio en cero cuando la recta es negativa:

$$\hat{y} = \max(wx + b, 0).$$

La función $\max(z, 0)$ se llama **ReLU** (rectified linear unit):

$$\mathrm{ReLU}(z) = \max(z, 0).$$

Si el score es negativo, la neurona da 0 (decimos que está "apagada"); si es positivo, deja pasar el valor sin cambiarlo (Figura 2). Hoy, la ReLU es la activación más usada en las capas internas de una red. Parece demasiado simple para servir de algo, pero, como veremos, lo único que se necesita es que la activación no sea lineal. Además, las redes con ReLU son mucho más fáciles de entrenar que las redes con sigmoide cuando tienen muchas capas.

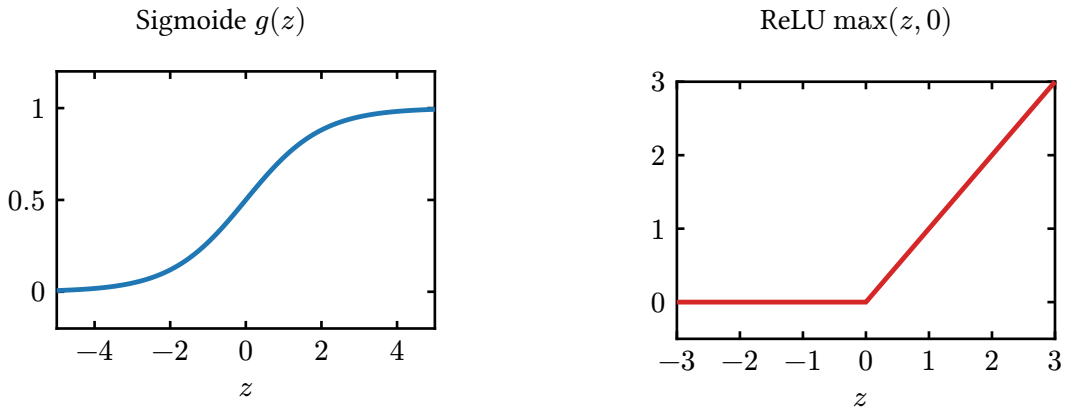



Figura 2: Las dos activaciones que usaremos. La sigmoide comprime cualquier número al intervalo $(0, 1)$, así que es útil en la salida cuando queremos una probabilidad. La ReLU es cero para $z < 0$ y la identidad para $z \geq 0$.

## 1.3. El límite de una neurona: XOR

Una sola neurona tiene la misma limitación que la regresión logística: su frontera de decisión, $\mathbf{w}^T\mathbf{x} + b = 0$, es una recta (o, con más features, un hiperplano). El ejemplo más simple de un problema que no se puede resolver con una recta es la función **XOR** ("o exclusivo"), en la que la etiqueta es 1 si exactamente una de las dos entradas es 1.

| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

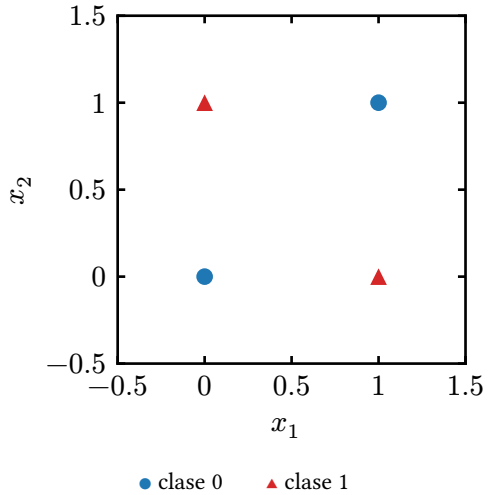



Figura 3: Los cuatro ejemplos de XOR. Ninguna recta deja los triángulos de un lado y los círculos del otro.

Ya vimos estos datos en la clase de árboles, donde un árbol los resuelve con dos preguntas. En la clase 4 resolvimos problemas parecidos con un feature map $\phi(\mathbf{x})$ diseñado a mano: para XOR serviría, por ejemplo, agregar la feature $x_1 x_2$. Pero diseñar features a mano requiere conocer bien el problema, y no escala. Con imágenes de miles de píxeles, nadie sabe qué features escribir.

> **Preguntar a la clase antes de mostrar la figura: "¿Alguna recta separa estos cuatro puntos?" Dejar que lo intenten en el aire. Recordar la figura de XOR de la clase de árboles.**

# 2. Parte 2: Redes neuronales

## 2.1. Apilar neuronas

Una red neuronal más compleja se construye tomando varias neuronas y "apilándolas": la salida de unas neuronas es la entrada de otras.

Veamos cómo resolver XOR de esta forma. Supongamos por un momento que sabemos algo del problema: la etiqueta es 1 cuando al menos una de las entradas es 1, excepto cuando las dos lo son. Esto sugiere calcular primero dos cantidades intermedias: cuántas entradas están en 1, y si las dos lo están. Cada una se puede calcular con una neurona con activación ReLU:

$$\begin{aligned} a_1 &= \mathrm{ReLU}(x_1 + x_2) && \text{(cuántas entradas están en 1)}, \\ a_2 &= \mathrm{ReLU}(x_1 + x_2 - 1) && \text{(vale 1 si las dos están en 1, y 0 si no)}. \end{aligned}$$

A partir de $a_1$ y $a_2$, la predicción es otra neurona, esta vez sin activación:

$$\hat{y} = a_1 - 2a_2.$$

Es decir, contamos las entradas que están en 1, y si son las dos, restamos 2. Esta es una red neuronal con tres neuronas (Figura 4). A las cantidades intermedias $a_1$ y $a_2$ se les llama **unidades ocultas** o neuronas ocultas, porque no aparecen en los datos: no son entradas ni etiquetas.

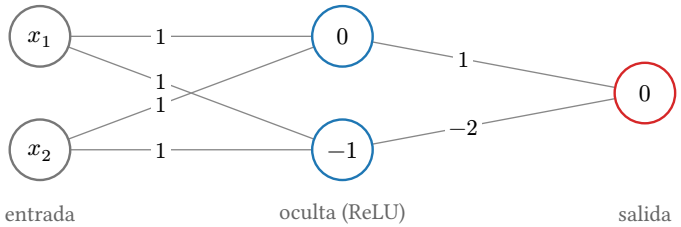



Figura 4: La red que resuelve XOR. Los números sobre las flechas son los pesos y los números dentro de cada neurona son los sesgos. Las neuronas ocultas usan ReLU y la de salida no tiene activación.

**Ejercicio 1**

Calcula la predicción de la red de la Figura 4 para los cuatro ejemplos de XOR y completa la tabla. ¿La red acierta en todos?

| $x_1$ | $x_2$ | $x_1 + x_2$ | $a_1$ | $a_2$ | $\hat{y} = a_1 - 2a_2$ |
|---|---|---|---|---|---|
| 0 | 0 | | | | |
| 0 | 1 | | | | |
| 1 | 0 | | | | |
| 1 | 1 | | | | |

**Ejercicio 1 en parejas, unos 12 min. Solución: $x_1 + x_2 = 0, 1, 1, 2$; $a_1 = 0, 1, 1, 2$; $a_2 = 0, 0, 0, 1$; $\hat{y} = 0, 1, 1, 0$. La red acierta en los cuatro. Señalar el caso (1, 1): $a_2$ se "enciende" y su peso $-2$ cancela a $a_1$.**

A calcular $\hat{y}$ a partir de $\mathbf{x}$, neurona por neurona y de izquierda a derecha, se le llama **propagación hacia adelante** (forward pass).

La ReLU parece un detalle. Veamos qué pasa si la quitamos.

**Ejercicio 2**

Repite el Ejercicio 1 sin la ReLU: $a_1 = x_1 + x_2$, $a_2 = x_1 + x_2 - 1$ y $\hat{y} = a_1 - 2a_2$.

1. Calcula $\hat{y}$ para los cuatro ejemplos. ¿Sigue resolviendo XOR?
2. Sustituye $a_1$ y $a_2$ en la fórmula de $\hat{y}$ y simplifica. ¿Qué tipo de función de $x_1$ y $x_2$ queda?

**Solución: $\hat{y} = 2, 1, 1, 0$, ya no resuelve XOR. Al simplificar: $\hat{y} = (x_1 + x_2) - 2(x_1 + x_2 - 1) = 2 - x_1 - x_2$, una función lineal. Que lo descubran ellos: esta es la idea 2 de la clase. En la sección 2.6 se generaliza.**

## 2.2. La capa oculta transforma los datos

¿Qué hacen exactamente las neuronas ocultas? Miremos los cuatro ejemplos a través de sus activaciones $(a_1, a_2)$ en lugar de sus entradas $(x_1, x_2)$. Los ejemplos $(0, 1)$ y $(1, 0)$ van al mismo punto, $(1, 0)$; el $(0, 0)$ va al $(0, 0)$, y el $(1, 1)$ va al $(2, 1)$. En este espacio nuevo, una recta sí separa las dos clases: es la frontera $a_1 - 2a_2 = 0.5$ de la neurona de salida (Figura 5).

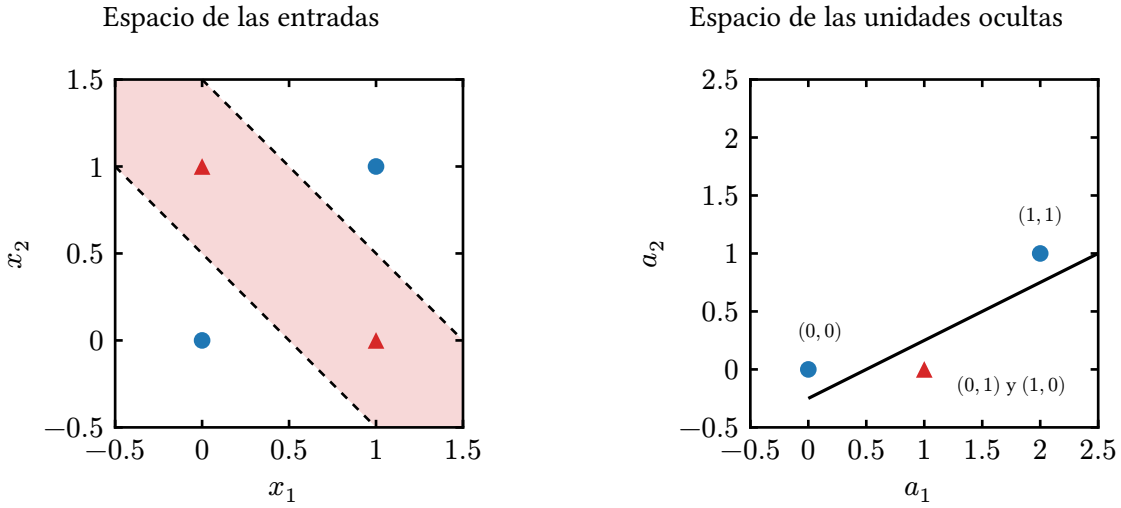



Figura 5: A la izquierda, la región donde la red predice $\hat{y} > 0.5$ (sombreada), en el espacio de las entradas: es la franja entre las rectas $x_1 + x_2 = 0.5$ y $x_1 + x_2 = 1.5$. A la derecha, los mismos ejemplos en el espacio de las unidades ocultas $(a_1, a_2)$. Los dos positivos caen en el mismo punto, $(1, 0)$, y la recta $a_1 - 2a_2 = 0.5$ de la neurona de salida separa las clases.

Es decir, las neuronas ocultas hacen lo mismo que hacíamos a mano en la clase 4: transforman los datos, $\phi(\mathbf{x}) = (a_1, a_2)$, de forma que un modelo lineal (la neurona de salida) pueda separarlos con una recta.

## 2.3. Redes totalmente conectadas

Construimos la red de XOR usando lo que sabíamos del problema: decidimos de antemano qué debía calcular cada neurona oculta. En la mayoría de las aplicaciones no tenemos ese conocimiento. Sería

más flexible y más general dejar que cada unidad oculta dependa de **todas** las entradas, con pesos que no fijamos nosotros:

$$a_1 = \text{ReLU}(\mathbf{w}_1^T \mathbf{x} + b_1), \quad \text{donde } \mathbf{w}_1 \in \mathbb{R}^2 \text{ y } b_1 \in \mathbb{R},$$
$$a_2 = \text{ReLU}(\mathbf{w}_2^T \mathbf{x} + b_2), \quad \text{donde } \mathbf{w}_2 \in \mathbb{R}^2 \text{ y } b_2 \in \mathbb{R}.$$

La red de XOR es un caso particular, con $\mathbf{w}_1 = \mathbf{w}_2 = (1, 1)$, $b_1 = 0$ y $b_2 = -1$. A una red así se le llama **totalmente conectada** (fully connected), porque cada unidad oculta está conectada con todas las entradas (Figura 6).

La red de XOR tiene dos **capas** de neuronas: la capa oculta, con $a_1$ y $a_2$, y la capa de salida, con $\hat{y}$. Generalicemos esta misma estructura a $d$ entradas y $m$ unidades ocultas. Una red totalmente conectada **de dos capas** con entradas $\mathbf{x} \in \mathbb{R}^d$ y $m$ unidades ocultas se define así. Para cada unidad oculta $j = 1, \ldots, m$,

$$z_j = \mathbf{w}_j^{(1)T} \mathbf{x} + b_j^{(1)}, \qquad a_j = \text{ReLU}(z_j), \qquad \text{donde } \mathbf{w}_j^{(1)} \in \mathbb{R}^d \text{ y } b_j^{(1)} \in \mathbb{R}.$$

Con las activaciones agrupadas en un vector $\mathbf{a} = (a_1, \ldots, a_m) \in \mathbb{R}^m$, la salida es

$$\hat{y} = \mathbf{w}^{(2)T} \mathbf{a} + b^{(2)}, \qquad \text{donde } \mathbf{w}^{(2)} \in \mathbb{R}^m \text{ y } b^{(2)} \in \mathbb{R}.$$

Los superíndices $(1)$ y $(2)$ distinguen los parámetros de las dos capas: los vectores $\mathbf{w}_j^{(1)}$ (uno por unidad oculta, cada uno en $\mathbb{R}^d$) y el vector $\mathbf{w}^{(2)}$ (en $\mathbb{R}^m$). En las clases anteriores usábamos el superíndice $(i)$ para numerar los ejemplos de entrenamiento. En estas notas casi siempre trabajamos con un solo ejemplo $\mathbf{x}$, así que el superíndice indicará la capa. Por ahora tenemos una sola capa oculta; en la sección 2.5 veremos que se pueden apilar tantas como queramos.

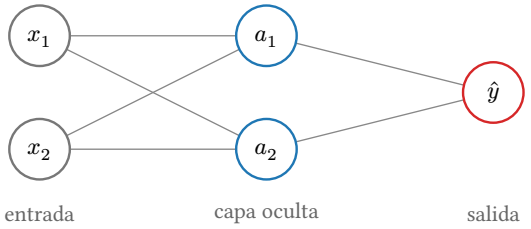



Figura 6: Una red totalmente conectada de dos capas, con 2 entradas, $m = 2$ unidades ocultas y una salida. Cada flecha tiene un peso, y cada unidad oculta y la salida tienen un sesgo.

La salida no tiene activación ReLU, porque en general queremos que pueda ser cualquier número. Si queremos una probabilidad, como en la regresión logística, le aplicamos la sigmoide: $\hat{y} = g(\mathbf{w}^{(2)T} \mathbf{a} + b^{(2)})$.

Igual que en regresión lineal y logística, los parámetros no los escogemos nosotros: los encuentra el entrenamiento, minimizando una función de pérdida. Lo veremos en la Parte 3.

> **Insistir: cada unidad oculta es una regresión logística más (con ReLU). No hay nada nuevo excepto conectarlas. Fin del Bloque 1 (aprox. 0:55). Descanso de 10 minutos.**

## 2.4. Notación vectorial

> **Bloque 2 (1:05).**

Antes de agregar más capas, vamos a escribir la red de dos capas de la sección anterior de forma más compacta, usando matrices y vectores. Hay además una razón práctica: la forma más directa de implementar la red es con un ciclo `for` sobre las unidades ocultas, pero con miles de unidades eso es

muy lento. Las librerías de álgebra lineal (y sobre todo las GPU) multiplican matrices muy rápido, así que conviene escribir todo como multiplicaciones de matrices.

Definimos la matriz de pesos $\mathbf{W}^{(1)} \in \mathbb{R}^{m \times d}$ poniendo los vectores $\mathbf{w}_j^{(1)}$ como filas:

$$\mathbf{W}^{(1)} = \begin{pmatrix} — \ \mathbf{w}_1^{(1)T} \ — \\ — \ \mathbf{w}_2^{(1)T} \ — \\ \vdots \\ — \ \mathbf{w}_m^{(1)T} \ — \end{pmatrix} \in \mathbb{R}^{m \times d}.$$

Por la definición del producto de una matriz por un vector, el vector $\mathbf{z} = (z_1, ..., z_m)$ se puede escribir como

$$\underbrace{\begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{pmatrix}}_{\mathbf{z} \in \mathbb{R}^{m \times 1}} = \underbrace{\begin{pmatrix} — \ \mathbf{w}_1^{(1)T} \ — \\ — \ \mathbf{w}_2^{(1)T} \ — \\ \vdots \\ — \ \mathbf{w}_m^{(1)T} \ — \end{pmatrix}}_{\mathbf{W}^{(1)} \in \mathbb{R}^{m \times d}} \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix}}_{\mathbf{x} \in \mathbb{R}^{d \times 1}} + \underbrace{\begin{pmatrix} b_1^{(1)} \\ b_2^{(1)} \\ \vdots \\ b_m^{(1)} \end{pmatrix}}_{\mathbf{b}^{(1)} \in \mathbb{R}^{m \times 1}},$$

o, de forma más breve,

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}.$$

Para obtener las activaciones aplicamos la ReLU a cada componente de $\mathbf{z}$ por separado. Si extendemos la notación de forma que, para un vector $\mathbf{z}$, $\mathrm{ReLU}(\mathbf{z})$ sea el vector con componentes $\mathrm{ReLU}\big(z_j\big)$, tenemos

$$\mathbf{a} = \mathrm{ReLU}(\mathbf{z}).$$

Definiendo de la misma forma $\mathbf{W}^{(2)} = \mathbf{w}^{(2)T} \in \mathbb{R}^{1 \times m}$, toda la red se resume en dos líneas:

$$\mathbf{a} = \mathrm{ReLU}\big(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big),$$
$$\hat{y} = \mathbf{W}^{(2)}\mathbf{a} + b^{(2)}.$$

A $\mathbf{W}^{(1)}$ y $\mathbf{W}^{(2)}$ se les llama matrices de pesos, y a $\mathbf{b}^{(1)}$ y $b^{(2)}$, sesgos. El par $\mathbf{W}^{(1)}, \mathbf{b}^{(1)}$ forma la primera **capa** de la red y $\mathbf{W}^{(2)}, b^{(2)}$ la segunda. Al vector de activaciones $\mathbf{a}$ se le llama la **capa oculta**. Cada capa hace lo mismo: multiplica su entrada por una matriz de pesos, suma un vector de sesgos y aplica la activación. En la siguiente sección usaremos esta observación para apilar más capas.

Por ejemplo, la red de XOR de la Figura 4 es

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad \mathbf{W}^{(2)} = \begin{pmatrix} 1 & -2 \end{pmatrix}, \quad b^{(2)} = 0.$$

Para $\mathbf{x} = (1, 1)$, el forward pass es

$$\mathbf{z} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}\begin{pmatrix} 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad \mathbf{a} = \mathrm{ReLU}(\mathbf{z}) = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad \hat{y} = \begin{pmatrix} 1 & -2 \end{pmatrix}\begin{pmatrix} 2 \\ 1 \end{pmatrix} + 0 = 0,$$

que es lo mismo que calculamos neurona por neurona en el Ejercicio 1.

## 2.5. Redes con más capas

Con esta notación es fácil apilar más capas. Sea $r$ el número de capas, y sean $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(r)}$ y $\mathbf{b}^{(1)}, \dots, \mathbf{b}^{(r)}$ las matrices de pesos y los sesgos de cada capa. Una red totalmente conectada con $r$ capas es

$$\begin{aligned}
\mathbf{a}^{(1)} &= \mathrm{ReLU}\big(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big), \\
\mathbf{a}^{(2)} &= \mathrm{ReLU}\big(\mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}\big), \\
&\vdots \\
\mathbf{a}^{(r-1)} &= \mathrm{ReLU}\big(\mathbf{W}^{(r-1)}\mathbf{a}^{(r-2)} + \mathbf{b}^{(r-1)}\big), \\
\hat{y} &= \mathbf{W}^{(r)}\mathbf{a}^{(r-1)} + \mathbf{b}^{(r)}.
\end{aligned}$$

Las dimensiones tienen que ser compatibles. Si la capa $k$ tiene $m_k$ unidades, es decir, $\mathbf{a}^{(k)} \in \mathbb{R}^{m_k}$, entonces $\mathbf{W}^{(k)}$ es de $m_k \times m_{k-1}$ y $\mathbf{b}^{(k)} \in \mathbb{R}^{m_k}$. Si escribimos además $\mathbf{a}^{(0)} = \mathbf{x}$ (con $m_0 = d$), cada capa oculta se calcula a partir de la anterior con la misma regla:

$$\mathbf{a}^{(k)} = \mathrm{ReLU}\big(\mathbf{W}^{(k)}\mathbf{a}^{(k-1)} + \mathbf{b}^{(k)}\big), \quad k = 1, \dots, r-1.$$

La última capa normalmente no lleva ReLU, para que la salida pueda ser negativa (o se le aplica la sigmoide o softmax, como veremos, si queremos probabilidades). A estas redes se les llama también **perceptrones multicapa** (MLP).

**Conteo de parámetros.** La capa $k$ tiene $m_k m_{k-1}$ pesos y $m_k$ sesgos, es decir, $(m_{k-1} + 1)m_k$ parámetros. Por ejemplo, consideremos una red para reconocer dígitos escritos a mano. Cada imagen tiene $28 \times 28 = 784$ píxeles, así que $d = 784$. Supongamos que la red tiene dos capas ocultas de 16 unidades y una salida con 10 unidades, una por dígito:

| Capa | Pesos | Sesgos | Total |
|---|---|---|---|
| 1 ($784 \to 16$) | $784 \cdot 16 = 12544$ | 16 | 12560 |
| 2 ($16 \to 16$) | $16 \cdot 16 = 256$ | 16 | 272 |
| 3 ($16 \to 10$) | $16 \cdot 10 = 160$ | 10 | 170 |
| **Total** | | | **13002** |

Entrenar esta red significa encontrar valores para esos 13002 números. Las redes que se usan hoy tienen millones o miles de millones de parámetros, pero la idea es la misma.

> **Hacer el conteo en la pizarra con ellos, capa por capa (es el ejercicio 3 del notebook).**

¿Para qué sirven varias capas? La intuición es que cada capa construye features a partir de las de la capa anterior. En imágenes, por ejemplo, las neuronas de la primera capa podrían detectar bordes pequeños, las de la segunda podrían combinar bordes en trazos, y las de la tercera combinar trazos en partes de un dígito, como un círculo o una línea vertical. Es una intuición útil, pero hay que tomarla con cuidado: en redes entrenadas de verdad, las neuronas casi nunca aprenden patrones tan limpios ni tan fáciles de interpretar.

## 2.6. ¿Por qué no usar la identidad como activación?

En el Ejercicio 2 vimos que, sin la ReLU, la red de XOR se convierte en una función lineal. Esto es cierto para cualquier red. Supongamos que la activación es la identidad, $\mathrm{ReLU}(z)$ reemplazada por $z$, y, para simplificar, que los sesgos son cero. Entonces, para una red de dos capas,

$$
\begin{aligned}
\hat{y} &= \mathbf{W}^{(2)}\mathbf{a}^{(1)} \\
&= \mathbf{W}^{(2)}\mathbf{z}^{(1)} \qquad \text{porque la activación es la identidad} \\
&= \mathbf{W}^{(2)}\mathbf{W}^{(1)}\mathbf{x} \\
&= \widetilde{\mathbf{W}}\mathbf{x}, \qquad \text{donde } \widetilde{\mathbf{W}} = \mathbf{W}^{(2)}\mathbf{W}^{(1)}.
\end{aligned}
$$

Las dos matrices se colapsan en una sola: aplicar una función lineal a otra función lineal da una función lineal de la entrada original. Con más capas pasa lo mismo, y con sesgos el resultado es $\widetilde{\mathbf{W}}\mathbf{x} + \tilde{\mathbf{b}}$. Una red de cien capas sin activaciones es equivalente a una sola neurona, y su frontera sigue siendo una recta. **La activación no lineal es lo que le da a la red su capacidad de representar funciones no lineales.**

## 2.7. La red aprende las features

En la clase 4 vimos que, con un feature map $\phi(\mathbf{x})$, un modelo lineal $\theta^T\phi(\mathbf{x})$ puede representar fronteras no lineales. El problema es escoger $\phi$: el desempeño depende mucho de esa elección, y normalmente se usa conocimiento del problema para diseñarla. A este proceso se le llama **ingeniería de features** (feature engineering).

Las redes neuronales se pueden ver como una forma de **aprender** el feature map. Llamemos $\beta$ a todos los parámetros de la red excepto los de la última capa. La última capa oculta, $\mathbf{a}^{(r-1)}$, es una función de $\mathbf{x}$ que depende de $\beta$; escribámosla $\phi_\beta(\mathbf{x})$. Entonces el modelo es

$$\hat{y} = \mathbf{W}^{(r)}\phi_\beta(\mathbf{x}) + \mathbf{b}^{(r)}.$$

Si $\beta$ estuviera fijo, esto sería un modelo lineal sobre las features $\phi_\beta(\mathbf{x})$, exactamente como en la clase 4. Pero al entrenar la red se optimizan a la vez $\beta$ y la última capa: no solo aprendemos un modelo lineal sobre las features, sino también las features mismas. Esto es lo que vimos en la Figura 5, ahora sin que nadie escoja los pesos. Por eso las redes neuronales dependen menos del conocimiento del problema. La contraparte es que las features que encuentra la red suelen ser difíciles de interpretar para una persona, y por eso a veces se dice que las redes son "cajas negras".

La Figura 7 muestra el efecto con datos más realistas. La regresión logística solo puede trazar una recta. Una red con una capa oculta de 3 unidades ReLU ya dobla la frontera, y con 32 unidades la frontera sigue la forma de los datos.

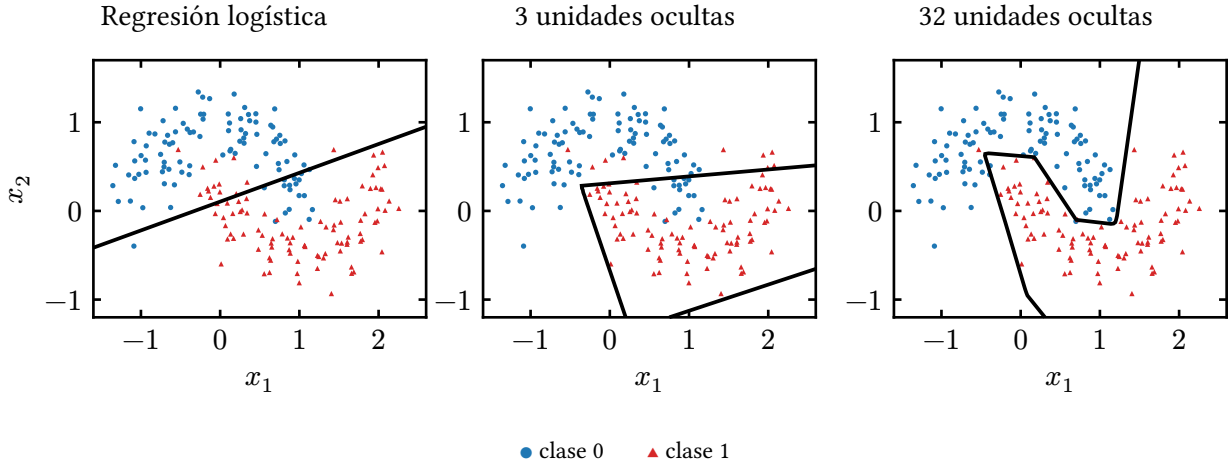



Figura 7: Fronteras de decisión ($\hat{y} = 0.5$) aprendidas sobre 200 ejemplos. A la izquierda, regresión logística (una neurona). En el centro y a la derecha, redes con una capa oculta de 3 y de 32 unidades ReLU (13 y 129 parámetros). La exactitud sobre datos de prueba es 86.8%, 88.6% y 95.9%.

## 2.8. Varias clases: softmax

Con dos clases basta una unidad de salida con sigmoide. Si hay $K$ clases, como los diez dígitos, la última capa tiene $K$ unidades, una por clase, con scores $z_1, \dots, z_K$. Para convertir los scores en probabilidades que sumen 1 se usa la función **softmax**:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K.$$

La exponencial hace que todos los valores sean positivos, y dividir entre la suma hace que sumen 1. La clase predicha es la de mayor probabilidad, que es también la de mayor score. Por ejemplo, con tres clases y scores $\mathbf{z} = (2, 1, -1)$, las probabilidades son aproximadamente $(0.71, 0.26, 0.04)$. Con dos clases, softmax da lo mismo que la sigmoide aplicada a la diferencia de los scores, así que softmax es la generalización de la sigmoide a más de dos clases.

> **5 minutos como máximo. Basta con que sepan que en MNIST la salida tiene 10 unidades y que softmax las convierte en probabilidades.**

## 2.9. (Opcional) Vectorización sobre varios ejemplos

> **OPCIONAL. Dar solo si el grupo entendió bien la notación vectorial y van a tiempo (unos 10 min). Si no, saltar a la demo del Playground y al lab. Lo importante para el lab es la última oración de la sección: PyTorch espera un ejemplo por fila.**

En la sección anterior vectorizamos sobre las unidades de una capa, pero seguimos procesando un ejemplo a la vez. En la práctica se procesan muchos ejemplos al mismo tiempo.

Pongamos $n$ ejemplos como filas de una matriz $\mathbf{X} \in \mathbb{R}^{n \times d}$, como en las clases anteriores. Para un ejemplo $\mathbf{x}$, la primera capa calcula $\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$. Transponiendo, $\mathbf{z}^T = \mathbf{x}^T\mathbf{W}^{(1)T} + \mathbf{b}^{(1)T}$, y como las filas de $\mathbf{X}$ son los $\mathbf{x}^T$, los scores de todos los ejemplos se obtienen con una sola multiplicación:

$$\underbrace{\mathbf{Z}^{(1)}}_{n \times m_1} = \underbrace{\mathbf{X}}_{n \times d} \ \underbrace{\mathbf{W}^{(1)T}}_{d \times m_1} + \underbrace{\mathbf{b}^{(1)T}}_{1 \times m_1}, \qquad \mathbf{A}^{(1)} = \mathrm{ReLU}\big(\mathbf{Z}^{(1)}\big).$$

La fila $i$ de $\mathbf{Z}^{(1)}$ contiene los scores de la primera capa para el ejemplo $i$. Las demás capas son iguales, reemplazando $\mathbf{X}$ por $\mathbf{A}^{(k-1)}$.

La suma del final tiene un detalle: $\mathbf{X}\mathbf{W}^{(1)T}$ es una matriz de $n \times m_1$, pero $\mathbf{b}^{(1)T}$ es una sola fila. Lo que queremos es sumarle esa fila a cada una de las $n$ filas, porque todos los ejemplos usan los mismos sesgos. NumPy y PyTorch lo hacen automáticamente, copiando la fila $n$ veces. A esto se le llama **broadcasting**. Los sesgos no se multiplican: siguen siendo $m_1$ parámetros.

En las notas de CS229 los ejemplos se ponen como columnas y la fórmula queda $\mathbf{Z} = \mathbf{W}\mathbf{X} + \mathbf{b}$; es lo mismo, transpuesto. Nosotros usamos filas porque es la convención de scikit-learn y de PyTorch: en PyTorch, `nn.Linear(d, m)` guarda una matriz de pesos de $m \times d$ y calcula exactamente `X @ W.T + b`.

> **Demo del Playground (unos 10 min). Antes de cada cambio, que la clase vote qué va a pasar: (1) dataset XOR, 2 neuronas ocultas, entrenar; (2) bajar a 1 neurona; (3) activación "Linear": la frontera vuelve a ser una recta (lo mismo del Ejercicio 2); (4) dataset espiral: ¿qué necesitamos? más capas y neuronas; (5) learning rate 3 o 10: la pérdida salta y no baja. Después: abrir labs/05/lab.ipynb, Parte 1 (ejercicios 1 a 3), unos 20 min.**

# 3. Parte 3: Cómo aprende una red

> **Bloque 3 (1:55). Mensaje central: no hay nada nuevo. Es el descenso por gradiente de la clase 2 aplicado a muchos parámetros a la vez.**

Hasta ahora escogimos los parámetros a mano. Con 13000 parámetros eso es imposible. Igual que en regresión lineal y logística, los vamos a encontrar minimizando una función de pérdida con descenso por gradiente.

## 3.1. La pérdida

La **pérdida** $L(\hat{y}, y)$ mide qué tan lejos está la predicción de la etiqueta en un ejemplo, y el **costo** $J$ es el promedio de la pérdida sobre los $n$ ejemplos de entrenamiento:

$$J = \frac{1}{n}\sum_{i=1}^{n} L\big(\hat{y}^{(i)}, y^{(i)}\big).$$

(Aquí, excepcionalmente, el superíndice $(i)$ numera los ejemplos.) La pérdida depende del problema, y ya las conocemos todas:

- **Regresión:** el error cuadrático, $L = (\hat{y} - y)^2$, como en regresión lineal. En PyTorch, `nn.MSELoss`.
- **Clasificación binaria:** la **entropía cruzada binaria**,

  $$L = -[y \log \hat{y} + (1 - y)\log(1 - \hat{y})],$$

  que es exactamente el negativo de la log-verosimilitud de un ejemplo en la regresión logística (clase 4). Allí maximizábamos la log-verosimilitud con ascenso por gradiente; aquí minimizamos su negativo con descenso por gradiente, que es lo mismo. En PyTorch, `nn.BCEWithLogitsLoss`, que aplica la sigmoide por nosotros.
- **Varias clases:** la **entropía cruzada**, $L = -\log \hat{y}_c$, donde $c$ es la clase correcta: queremos que la probabilidad que la red le asigna a la clase correcta sea alta. En PyTorch, `nn.CrossEntropyLoss`, que aplica softmax por nosotros.

## 3.2. Descenso por gradiente

Agrupemos todos los pesos y sesgos de la red en un vector $\theta$. El descenso por gradiente repite

$$\theta := \theta - \alpha \nabla_\theta J,$$

donde $\alpha$ es la tasa de aprendizaje. Cada componente del gradiente dice cuánto cambia el costo si movemos un poco ese parámetro, y en qué dirección. Restar el gradiente mueve cada parámetro en la dirección en la que el costo baja. En la práctica se usa la versión con mini-batches: en cada paso, el gradiente se calcula con un grupo pequeño de ejemplos escogidos al azar.

Antes de ver cómo se calcula el gradiente de una red, hagamos algunos pasos a mano con la red más pequeña posible: una neurona sin activación, sin sesgo y con un solo peso, $\hat{y} = wx$, un solo ejemplo $x = 2$, $y = 10$, y pérdida $L = (\hat{y} - y)^2$. Para encontrar la derivada de $L$ respecto a $w$, notemos que $L$ no depende de $w$ directamente, sino a través de $\hat{y}$: el peso determina la predicción, y la predicción determina la pérdida. En estos casos se usa la **regla de la cadena**: si $L$ depende de $\hat{y}$ y $\hat{y}$ depende de $w$, entonces

$$\frac{dL}{dw} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dw}.$$

Es decir, cuánto cambia $L$ al mover $w$ es cuánto cambia $L$ al mover $\hat{y}$, multiplicado por cuánto cambia $\hat{y}$ al mover $w$. Calculemos cada factor por separado. Como $L = (\hat{y} - y)^2$ y $\hat{y} = wx$,

$$\frac{dL}{d\hat{y}} = 2(\hat{y} - y), \qquad \frac{d\hat{y}}{dw} = x.$$

Multiplicando,

$$\frac{dL}{dw} = 2(\hat{y} - y)x.$$

En nuestro ejemplo, con $x = 2$, $\frac{dL}{dw} = 4(\hat{y} - 10)$. Vamos a volver a usar esta misma idea en la siguiente sección: backpropagation no es más que la regla de la cadena aplicada a muchos pasos seguidos.

**Ejercicio 3**

Empezando con $w = 1$ y $\alpha = 0.05$, completa tres pasos de descenso por gradiente.

| Paso | $w$ | $\hat{y} = 2w$ | $L = (\hat{y} - 10)^2$ | $\frac{dL}{dw} = 4(\hat{y} - 10)$ |
|---|---|---|---|---|
| 0 | 1 | | | |
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |

¿Hacia qué valor se acerca $w$? ¿Por qué?

**Solución:** $w = 1, 2.6, 3.56, 4.136$; $\hat{y} = 2, 5.2, 7.12, 8.272$; $L = 64, 23.04, 8.29, 2.99$; **gradiente** $= -32, -19.2, -11.52, -6.91$. **Se acerca a** $w = 5$, **porque** $2 \cdot 5 = 10$. **El primer gradiente, –32, lo vamos a volver a ver en el lab: PyTorch lo calcula solo.**

La Figura 8 muestra cómo evoluciona $w$ con dos tasas de aprendizaje. Con $\alpha = 0.05$, $w$ se acerca a 5 poco a poco. Con $\alpha = 0.3$, cada paso se pasa del mínimo, cada vez más lejos, y el descenso diverge. Es lo mismo que vimos en la clase 2, y es el error más común al entrenar una red: si la pérdida sube o se vuelve `nan`, lo primero es bajar la tasa de aprendizaje.

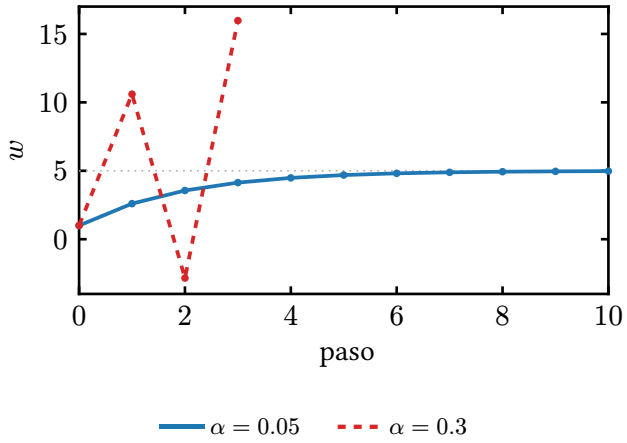



Figura 8: El peso $w$ en los primeros pasos de descenso por gradiente, con $\alpha = 0.05$ (converge a 5) y con $\alpha = 0.3$ (oscila alrededor de 5, cada vez más lejos).

### 3.3. Backpropagation: el error viaja hacia atrás

Con una neurona, la derivada sale directo. En una red, en cambio, un peso de la primera capa afecta a la pérdida a través de todas las capas que vienen después. **Backpropagation** (propagación hacia atrás) es el algoritmo que calcula todas esas derivadas de forma eficiente. No es un método de aprendizaje distinto: el aprendizaje sigue siendo descenso por gradiente, y backpropagation es solo la forma de calcular el gradiente.

Veamos la idea con una red de dos pesos: una unidad oculta con ReLU y una salida sin activación,

$$a = \text{ReLU}(w_1 x), \qquad \hat{y} = w_2 a, \qquad L = (\hat{y} - y)^2,$$

con $x = 2$, $y = 10$, $w_1 = 1$ y $w_2 = 1$. El forward pass da $a = 2$, $\hat{y} = 2$ y $L = 64$.

Ahora vamos de derecha a izquierda, preguntándonos en cada paso cuánto cambia la pérdida si cambia ese valor:

1. **En la salida.** $\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = -16$. Esta es la "señal de error". Es negativa porque la predicción es demasiado pequeña: si $\hat{y}$ aumenta, la pérdida baja.
2. **El peso $w_2$.** Como $\hat{y} = w_2 a$, mover $w_2$ mueve $\hat{y}$ en proporción a $a$. Entonces $\frac{\partial L}{\partial w_2} = -16 \cdot a = -32$.
3. **La unidad oculta.** Mover $a$ mueve $\hat{y}$ en proporción a $w_2$. La señal de error llega a $a$ multiplicada por el peso de la conexión: $\frac{\partial L}{\partial a} = -16 \cdot w_2 = -16$.
4. **La ReLU.** Como $w_1 x = 2 > 0$, la ReLU deja pasar los cambios tal cual (su pendiente es 1). Si la neurona estuviera apagada ($w_1 x < 0$), la pendiente sería 0 y la señal se detendría aquí.
5. **El peso $w_1$.** Igual que en el paso 2: $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot 1 \cdot x = -16 \cdot 2 = -32$.

Ese es todo el algoritmo. El error se calcula en la salida y viaja hacia atrás, capa por capa. Al cruzar cada conexión se multiplica por su peso, y al cruzar cada activación se multiplica por su pendiente. Cuando la señal llega a un peso, la derivada de ese peso es la señal multiplicada por la entrada que recibe ese peso. Es la regla de la cadena, aplicada de forma ordenada y reutilizando los cálculos que ya hicimos.

Hay una forma intuitiva de leer esto. Si la salida debería ser más grande, cada conexión "pide" un cambio en la capa anterior, en la dirección que más ayudaría a la salida, y esos pedidos se pasan hacia atrás, capa por capa. Como la derivada de un peso es proporcional a la entrada que recibe, las conexiones que vienen de neuronas muy activas son las que más influyen en la salida, y por eso son las que más cambian. Las que vienen de neuronas apagadas no cambian.

**Ejercicio 4**

Para la red de dos pesos de arriba ($x = 2$, $y = 10$), calcula el forward pass y las dos derivadas $\frac{\partial L}{\partial w_1}$ y $\frac{\partial L}{\partial w_2}$ si empezamos con $w_1 = -1$ y $w_2 = 1$. ¿Qué pasa con $w_1$ si damos un paso de descenso por gradiente? ¿Por qué?

**Solución: $w_1 x = -2$, así que $a = 0$, $\hat{y} = 0$, $L = 100$. La señal de error es $2(0 - 10) = -20$. $\frac{\partial L}{\partial w_2} = -20 \cdot a = 0$ y $\frac{\partial L}{\partial w_1} = -20 \cdot w_2 \cdot 0 \cdot x = 0$, porque la ReLU está apagada. Ningún peso cambia: la neurona está "muerta" para este ejemplo. Es una buena excusa para explicar por qué en el lab usamos más neuronas de las estrictamente necesarias.**

En la práctica nadie calcula estas derivadas a mano. Las librerías como PyTorch registran las operaciones del forward pass y, con una sola instrucción, recorren el camino hacia atrás y calculan el

gradiente de la pérdida respecto a cada parámetro. A esto se le llama **diferenciación automática** (autograd).

## 3.4. Entrenar una red en PyTorch

Un entrenamiento en PyTorch tiene siempre la misma estructura, con cinco líneas que se repiten en cada paso:

```python
modelo = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 1))
perdida = nn.BCEWithLogitsLoss()
optimizador = torch.optim.SGD(modelo.parameters(), lr=0.1)

for paso in range(1000):
    optimizador.zero_grad()   # 1. borrar los gradientes anteriores
    y_hat = modelo(X)         # 2. forward pass
    L = perdida(y_hat, y)     # 3. calcular la pérdida
    L.backward()              # 4. backpropagation
    optimizador.step()        # 5. un paso de descenso
```

La Figura 9 muestra la pérdida durante el entrenamiento de una red con una capa oculta de 32 unidades sobre los datos de la Figura 7. Baja rápido al principio y luego cada vez más despacio, como en regresión lineal. Igual que con los árboles, que la pérdida de entrenamiento baje no garantiza que la red generalice: una red grande también puede memorizar los datos. Eso lo veremos la próxima clase.

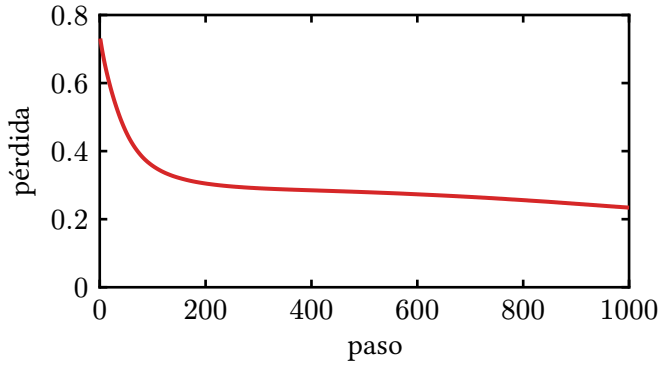



Figura 9: Entropía cruzada de entrenamiento de una red con 32 unidades ocultas sobre los datos de la Figura 7, en función del número de pasos de descenso por gradiente ($\alpha = 0.1$). La exactitud final sobre datos de prueba es 90.4%.

> **Ejercicio 4 en papel (aprox. 10 min) y después el lab, Parte 2 (ejercicios 4 y 5 del notebook, unos 25 min). El momento clave: `w.grad` da −32, lo mismo que calcularon a mano en el Ejercicio 3. Hacerlo notar en voz alta.**

## 3.5. (Opcional) Backpropagation en general

> **OPCIONAL, para leer en casa. No darlo en clase.**

Para una red con $r$ capas, llamemos $\boldsymbol{\delta}^{(k)} = \partial L / \partial \mathbf{z}^{(k)}$ a la señal de error que llega a los scores de la capa $k$, donde $\mathbf{z}^{(k)} = \mathbf{W}^{(k)}\mathbf{a}^{(k-1)} + \mathbf{b}^{(k)}$. Backpropagation calcula, para un ejemplo:

1. En la salida, con sigmoide y entropía cruzada binaria (o con softmax y entropía cruzada): $\boldsymbol{\delta}^{(r)} = \hat{\mathbf{y}} - \mathbf{y}$. Es el mismo $h - y$ que apareció en el gradiente de la regresión logística en la clase 4.
2. Para cada capa, de la última a la primera:

$$\frac{\partial L}{\partial \mathbf{W}^{(k)}} = \boldsymbol{\delta}^{(k)} \mathbf{a}^{(k-1)T}, \qquad \frac{\partial L}{\partial \mathbf{b}^{(k)}} = \boldsymbol{\delta}^{(k)}, \qquad \boldsymbol{\delta}^{(k-1)} = \left(\mathbf{W}^{(k)T} \boldsymbol{\delta}^{(k)}\right) \odot g'\left(\mathbf{z}^{(k-1)}\right),$$

donde $\odot$ es el producto elemento a elemento y $g'$ es la derivada de la activación.

Son las mismas tres reglas del ejemplo de dos pesos, escritas para capas completas: la derivada de un peso es la señal que llega multiplicada por la entrada que recibe; al cruzar las conexiones, la señal se multiplica por los pesos ($\mathbf{W}^T$); y al cruzar la activación, se multiplica por su pendiente ($g'$). Para la ReLU, $g'(z)$ es 1 si $z > 0$ y 0 si no.

## 4. Resumen

Las tres ideas de la clase:

- **Una neurona es una regresión logística:** una suma ponderada de las entradas más un sesgo, $z = \mathbf{w}^T \mathbf{x} + b$, seguida de una activación, $a = g(z)$. Las activaciones más usadas son la sigmoide y la ReLU, $\max(z, 0)$.
- **Apilar neuronas con activaciones no lineales produce fronteras no lineales.** Las capas ocultas transforman los datos en features nuevas, y la última capa es un modelo lineal sobre esas features: la red aprende el $\phi(\mathbf{x})$ que en la clase 4 diseñábamos a mano. Sin activaciones no lineales, cualquier red, por grande que sea, es equivalente a un modelo lineal.
- **Una red aprende con descenso por gradiente, y backpropagation es la forma de calcular el gradiente.** El error se calcula en la salida y viaja hacia atrás capa por capa, multiplicándose por los pesos y por las pendientes de las activaciones. En PyTorch lo hace `loss.backward()`.

Además:

- Con notación vectorial, cada capa calcula $\mathbf{a}^{(k)} = \mathrm{ReLU}\left(\mathbf{W}^{(k)} \mathbf{a}^{(k-1)} + \mathbf{b}^{(k)}\right)$, donde la fila $j$ de $\mathbf{W}^{(k)}$ son los pesos de la unidad $j$. Una capa con $m_{k-1}$ entradas y $m_k$ unidades tiene $(m_{k-1} + 1)m_k$ parámetros.
- Con varias clases, la salida tiene una unidad por clase y softmax convierte los scores en probabilidades.
- La pérdida es el error cuadrático en regresión y la entropía cruzada en clasificación (el negativo de la log-verosimilitud).

> **Cierre (2:50): leer las tres ideas en voz alta y pedir a dos o tres alumnos que las expliquen con sus palabras. Presentar la Tarea 4.**